TP 1 : Outils de manipulation et de traitement d'images en Python

L'objectif de ce TP est de se familiariser avec les bibliothèques de traitement, manipulation et visualisation d'images en Python: OpenCV, Matplotlib et Numpy

Prérequis : Python et Pip installés.

Utiliser le système de gestion de package Pip pour installer toutes les bibliothèques susmentionnées.

1. Installation : pip install opencv-python

   pip install numpy

   pip install matplotlib

2. Utilisation de OpenCV:

   - Le code suivant permet de lire, afficher une image et accéder à ses pixels, en utilisant la bibliothèque OpenCV. Si l'image est en niveaux de gris (grayscale), image.shape aura seulement deux valeurs : (height, width).

```
import cv2
# Read the image
img = cv2.imread('cablecar.bmp')
if image is None:
  print("Erreur : Impossible de charger l'image.")
  else:
    print (type(img))
    # Get the image height and width
    height, width, channels = img.shape
    print (f'height: {height} / width: {width} / channels: {channels}')
    # Create a window with a title "Image" to display the image
    cv2.imshow("Image", img)
    # Hold the screen until the user closes it.
    cv2.waitKey(0)
    # Destroy the window
    cv2.destroyAllWindows()
```

   - Le code suivant nous montre comment redimensionner et sauvegarder une image avec OpenCV.

```
# Resize an image
resized_img = cv2.resize(img, (int(img.shape[1]/2), int(img.shape[0]/2)))
# Save the image
cv2.imwrite("resized.png", resized_img)
```

- Il existe des espaces couleur autres que l'espace RGB, tels que les espaces YCrCb et HSV. Le code suivant permet de faire les changements d'espaces colorimétriques ainsi qu'en niveau de gris. La bibliothèque numpy est utilisée pour concaténer les images afin de les afficher côte à côte.

```
import numpy as np
#...

# Convert from BGR to YCrCb color space: Y is the Luminance component, Cb and Cr chrominance components
img_ycrcb = cv2.cvtColor(resized_img, cv2.COLOR_BGR2YCrCb)
cv2.imwrite("ycrcb_random.png", img_ycrcb)
# Convert from BGR to HSV color space
img_hsv = cv2.cvtColor(resized_img, cv2.COLOR_BGR2HSV)
cv2.imwrite("hsv_random.png", img_hsv)
# Convert an RGB image to grayscale
img_gray = cv2.cvtColor(resized_img, cv2.COLOR_BGR2GRAY)
# concatenate image Horizontally
images = np.concatenate((cv2.cvtColor(img_gray, cv2.COLOR_GRAY2BGR), img_ycrcb, img_hsv), axis=1)
cv2.imshow('Gray, YCrCb and HSV images', images)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

3. Utilisation de la bibliothèque Matplotlib pour l'affichage:
   - Il faut d'abord convertir l'image du BGR (l'espace couleur utilisé par défaut lors de la lecture d'une image avec OpenCV) au RGB (l'espace couleur utilisé pour l'affichage des images avec Matplotlib)

```
import matplotlib.pyplot as plt
# Read the image with OpenCV

# Converting BGR color to RGB color format
RGB_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Display the image using plt
plt.imshow(RGB_img)
# Show without the axes
plt.axis('off')
# Put a title
plt.title('Image')
plt.show()
```

- Avec OpenCV, on peut séparer les 3 canaux (Blue, Red, Green) d'une image couleur en 3 images différentes où chaque image va contenir les valeurs des pixels d'un canal, comme on peut regrouper les 3 images des différents canaux afin de former une image RGB.

```
# %%
# Separate and merge RGB images
B, G, R = cv2.split(img)
RGB_img = cv2.merge([R, G, B])
```

```
plt.imshow(RGB_img)
plt.axis('off')
plt.title('Image')
plt.show()
```

- Affichage des images qui représentent les canaux R, G et B avec Matplotlib

```
# %%
# création d'images vide de la même forme que l'image d'origine
R_img = np.zeros_like(img)
G_img = np.zeros_like(img)
B_img = np.zeros_like(img)

# assigner la valeur de chaque canal a la composante correspondante en laissant les autres canaux a 0
R_img[:, :, 0] = R  # Canal Rouge
G_img[:, :, 1] = G  # Canal Vert
B_img[:, :, 2] = B  # Canal Bleu

figure, plot = plt.subplots(nrows=1, ncols=3)

plot[0].imshow(R_img)
plot[0].set_title('canal rouge')
plot[0].axis('off')

plot[1].imshow(G_img)
plot[1].set_title('canal vert')
plot[1].axis('off')

plot[2].imshow(B_img)
plot[2].set_title('canal bleu')
plot[2].axis('off')

plt.show()
```

4. Ajouter des bordures à une image

La fonction pad de NumPy permet d'ajouter du remplissage (padding) autour d'un tableau (numpy.ndarray). Elle est couramment utilisée pour ajouter des marges autour d'une image ou bien pour remplir une matrice avec des valeurs supplémentaires.

```
# %%
# Ajout d'un padding de 2 pixels de large avec une valeur de 0
padded_image = np.pad(RGB_img, pad_width=((10, 10), (10, 10), (0, 0)), mode='constant', constant_values=0)

figure, plot = plt.subplots(nrows=1, ncols=2)

plot[0].imshow(RGB_img)
plot[0].set_title('l'image originale')
plot[0].axis('off')
```

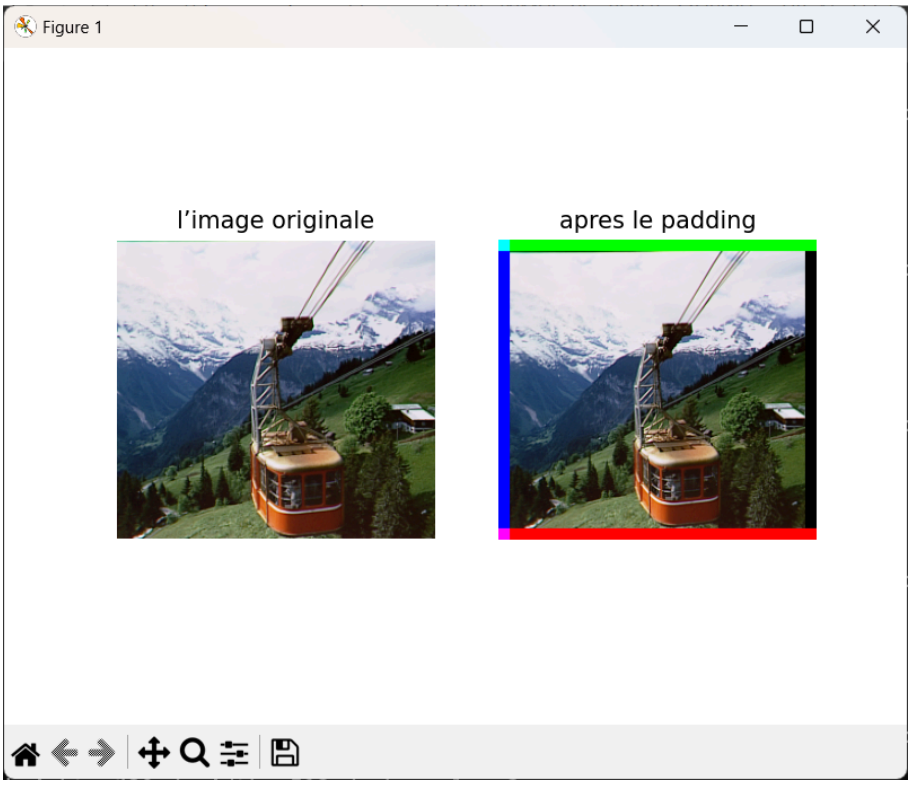
```
plot[1].imshow(padded_image)
plot[1].set_title('apres le padding')
plot[1].axis('off')

plt.show()
```

5. Accéder à une ligne ou une colonne spécifique :

Avec NumPy, on peut accéder à une ligne ou une colonne spécifique d'une image en utilisant l'indexation. Une image est souvent représentée sous la forme d'un tableau NumPy de forme (hauteur, largeur, canaux), où :
hauteur = nombre de lignes
largeur = nombre de colonnes
canaux = nombre de canaux (ex. : 3 pour une image RGB)

Exemples:
ligne = image[index, : , :]
colonne = image[: , index, :]
image[: , index, 1] = 0

<u>Exercice:</u>

- Utiliser Matplotlib pour afficher les trois images: niveau de gris, YCbCr et HSV à partir de l'image originale "cablecar.bmp".
- Créer une bordure telle montrée dans la figure ci dessous